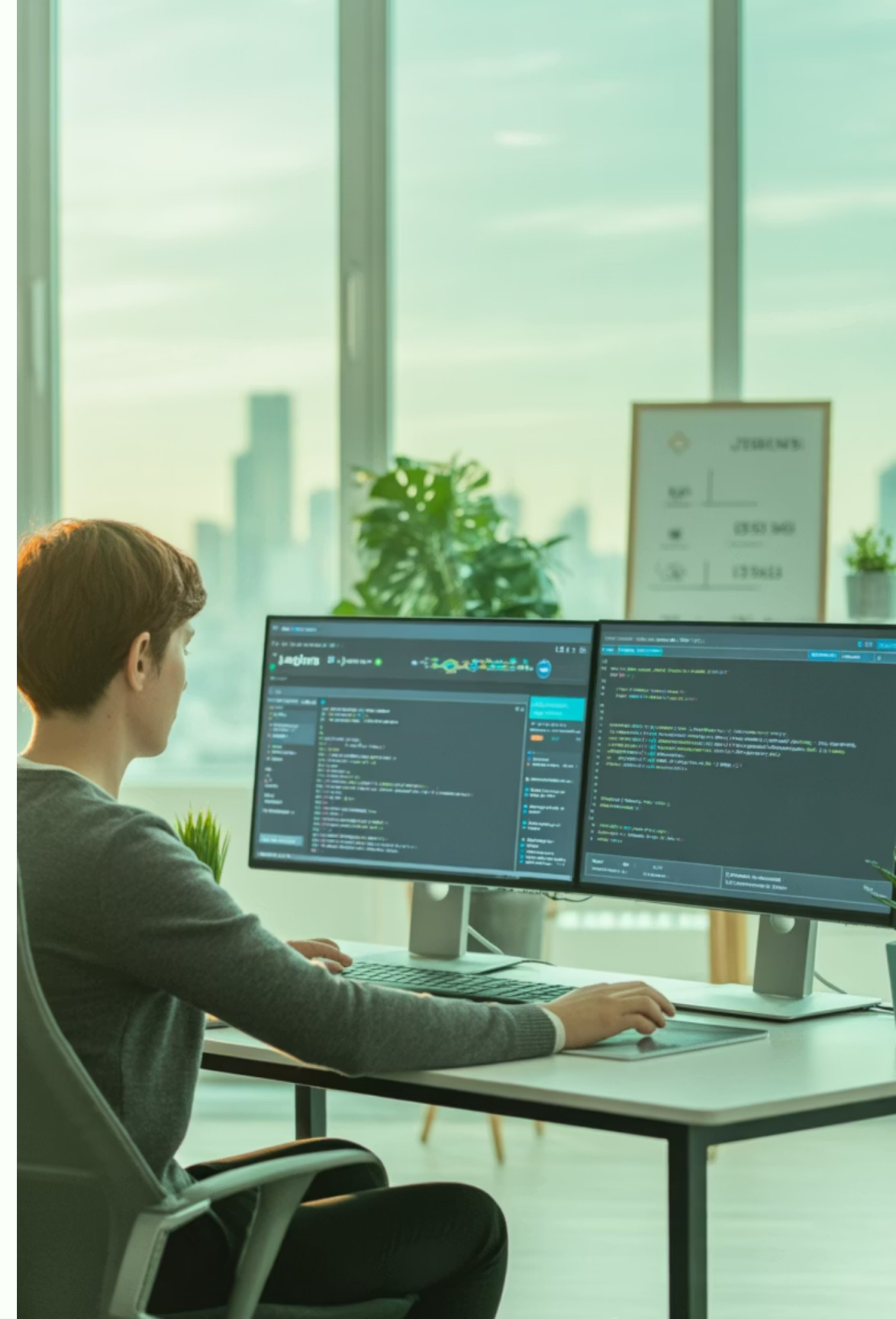


Jenkins Server Setup

A comprehensive guide to understanding, installing, and configuring Jenkins for your CI/CD pipeline



Agenda

1

Jenkins Architecture

Understanding the Master-Agent model

2

Prerequisites

Required software and system specifications

3

Installation Options

Local, Docker, and cloud-based approaches

4

Setup & Configuration

Initial wizard, dashboard tour, and security basics

Jenkins Architecture: Master-Agent Model

Jenkins follows a Master-Agent architecture that enables distributed builds and testing across multiple machines.

The **Master** coordinates jobs, schedules builds, and manages the UI

The **Agents** (formerly called slaves) execute the actual build tasks

- This distributed approach allows for parallel execution and specialized build environments



Prerequisites for Jenkins Installation

Java Requirements

- Java 11 or Java 17 (LTS releases)
- OpenJDK or Oracle JDK
- JAVA_HOME environment variable set

Version Control

- Git client installed
- Access to repositories

System Requirements

- 256MB RAM (minimum)
- 1GB+ RAM (recommended)
- 10GB+ disk space
- Stable network connection

These requirements may vary based on your specific workload and the number of builds you plan to run simultaneously.

Installation Option 1: Local Installation

Platform-Specific Instructions

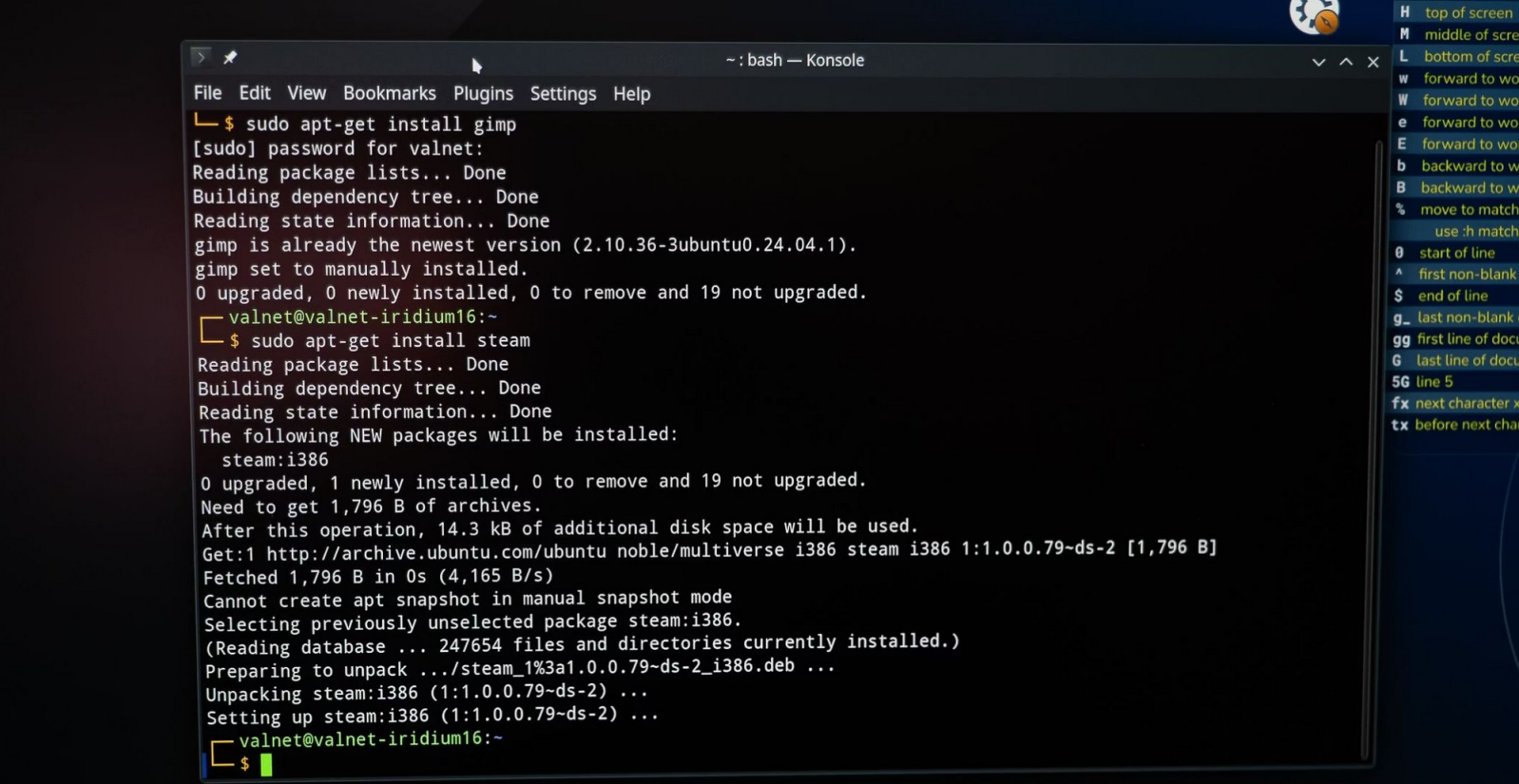
Jenkins can be installed directly on your operating system:

Linux: via package managers (apt, yum)

Windows: MSI installer or as a service

macOS: Homebrew or installer package

Best for: Development environments, small teams, or when you need direct access to local resources.



```
~ : bash — Konsole
File Edit View Bookmarks Plugins Settings Help
└─$ sudo apt-get install gimp
[sudo] password for valnet:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
gimp is already the newest version (2.10.36-3ubuntu0.24.04.1).
gimp set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 19 not upgraded.
valnet@valnet-iridium16:~$ sudo apt-get install steam
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  steam:i386
0 upgraded, 1 newly installed, 0 to remove and 19 not upgraded.
Need to get 1,796 B of archives.
After this operation, 14.3 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/multiverse i386 steam i386 1:1.0.0.79-ds-2 [1,796 B]
Fetched 1,796 B in 0s (4,165 B/s)
Cannot create apt snapshot in manual snapshot mode
Selecting previously unselected package steam:i386.
(Reading database ... 247654 files and directories currently installed.)
Preparing to unpack .../steam_1%3a1.0.0.79-ds-2_i386.deb ...
Unpacking steam:i386 (1:1.0.0.79-ds-2) ...
Setting up steam:i386 (1:1.0.0.79-ds-2) ...
valnet@valnet-iridium16:~$
```

Installation Option 2: Dockerized Jenkins

Running Jenkins in Docker offers several advantages:

- Consistent environment across different platforms
- Easy upgrades and rollbacks
- Isolation from the host system
- Simple configuration via Docker Compose

Basic command to get started:

```
docker run -p 8080:8080 -p 50000:50000 \-v  
jenkins_home:/var/jenkins_home \jenkins/jenkins:lts
```



Installation Option 3: Cloud-Based Jenkins

AWS

- EC2 instance with Jenkins AMI
- Elastic Beanstalk deployment
- Integration with AWS services

Azure

- Azure VM deployment
- Azure DevOps integration
- Managed identity support

GCP

- Compute Engine instances
- Google Kubernetes Engine
- Cloud Build integration

Cloud-based installations are ideal for teams requiring scalability, high availability, and integration with cloud-native services. They typically offer better performance for distributed teams.

Hands-on Demo: Installation Process

We'll walk through a Linux-based installation as our example:

1. Update package repositories

Install Java: `sudo apt install openjdk-11-jdk`

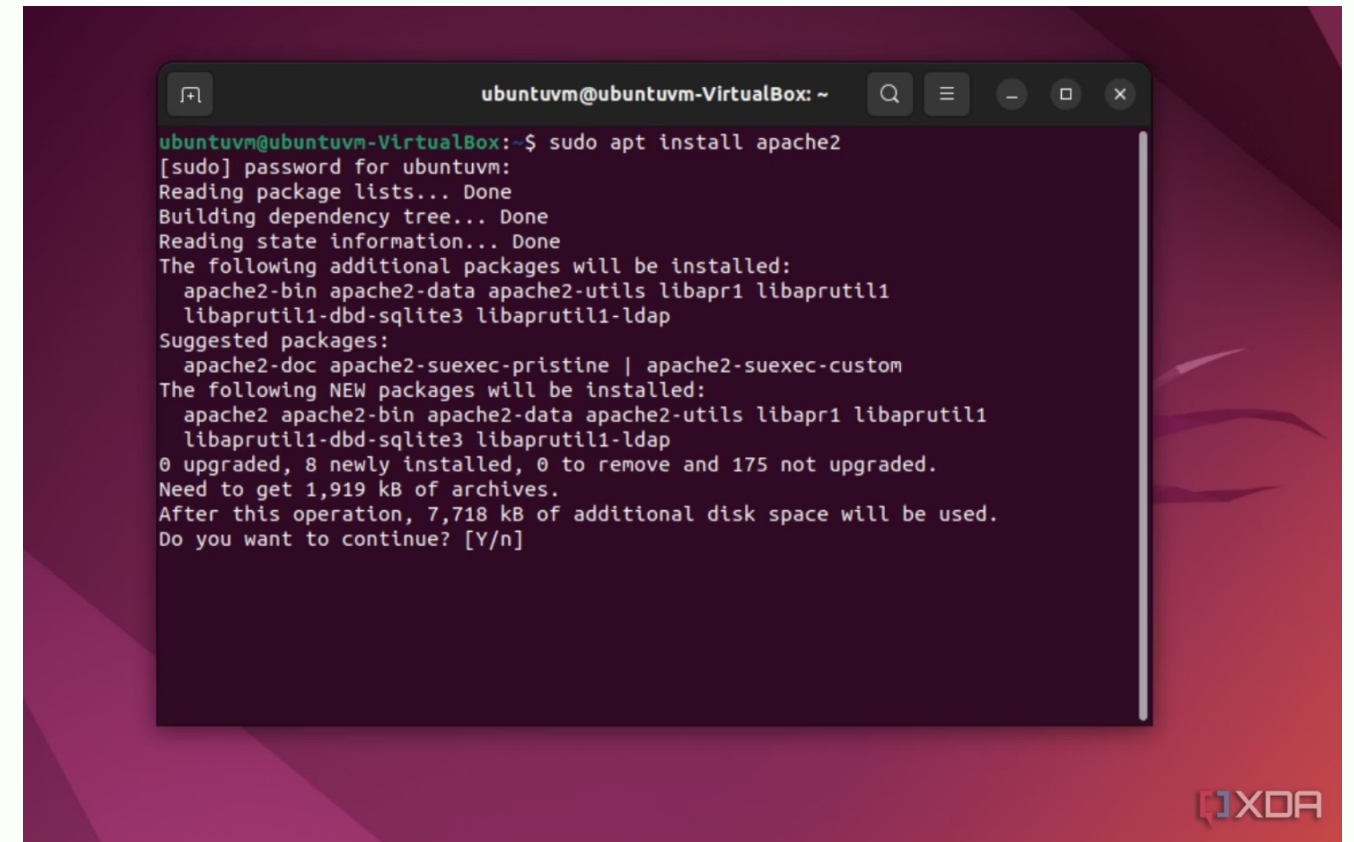
3. Add Jenkins repository key

4. Add Jenkins repository to sources list

Install Jenkins: `sudo apt install jenkins`

Start Jenkins service: `sudo systemctl start jenkins`

Enable auto-start: `sudo systemctl enable jenkins`

A terminal window titled 'ubuntuvm@ubuntuvm-VirtualBox: ~' showing the command 'sudo apt install apache2' and its output. The output indicates that several additional packages will be installed along with apache2, including apache2-bin, apache2-data, apache2-utils, libapr1, libaprutil1, libaprutil1-dbd-sqlite3, and libaprutil1-ldap. It also shows the disk space requirements for the installation.

```
ubuntuvm@ubuntuvm-VirtualBox: ~  
ubuntuvm@ubuntuvm-VirtualBox:~$ sudo apt install apache2  
[sudo] password for ubuntuvm:  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
The following additional packages will be installed:  
  apache2-bin apache2-data apache2-utils libapr1 libaprutil1  
  libaprutil1-dbd-sqlite3 libaprutil1-ldap  
Suggested packages:  
  apache2-doc apache2-suexec-pristine | apache2-suexec-custom  
The following NEW packages will be installed:  
  apache2 apache2-bin apache2-data apache2-utils libapr1 libaprutil1  
  libaprutil1-dbd-sqlite3 libaprutil1-ldap  
0 upgraded, 8 newly installed, 0 to remove and 175 not upgraded.  
Need to get 1,919 kB of archives.  
After this operation, 7,718 kB of additional disk space will be used.  
Do you want to continue? [Y/n]
```

Initial Setup Wizard

After installation, Jenkins guides you through initial configuration:

Access Jenkins at `http://localhost:8080`

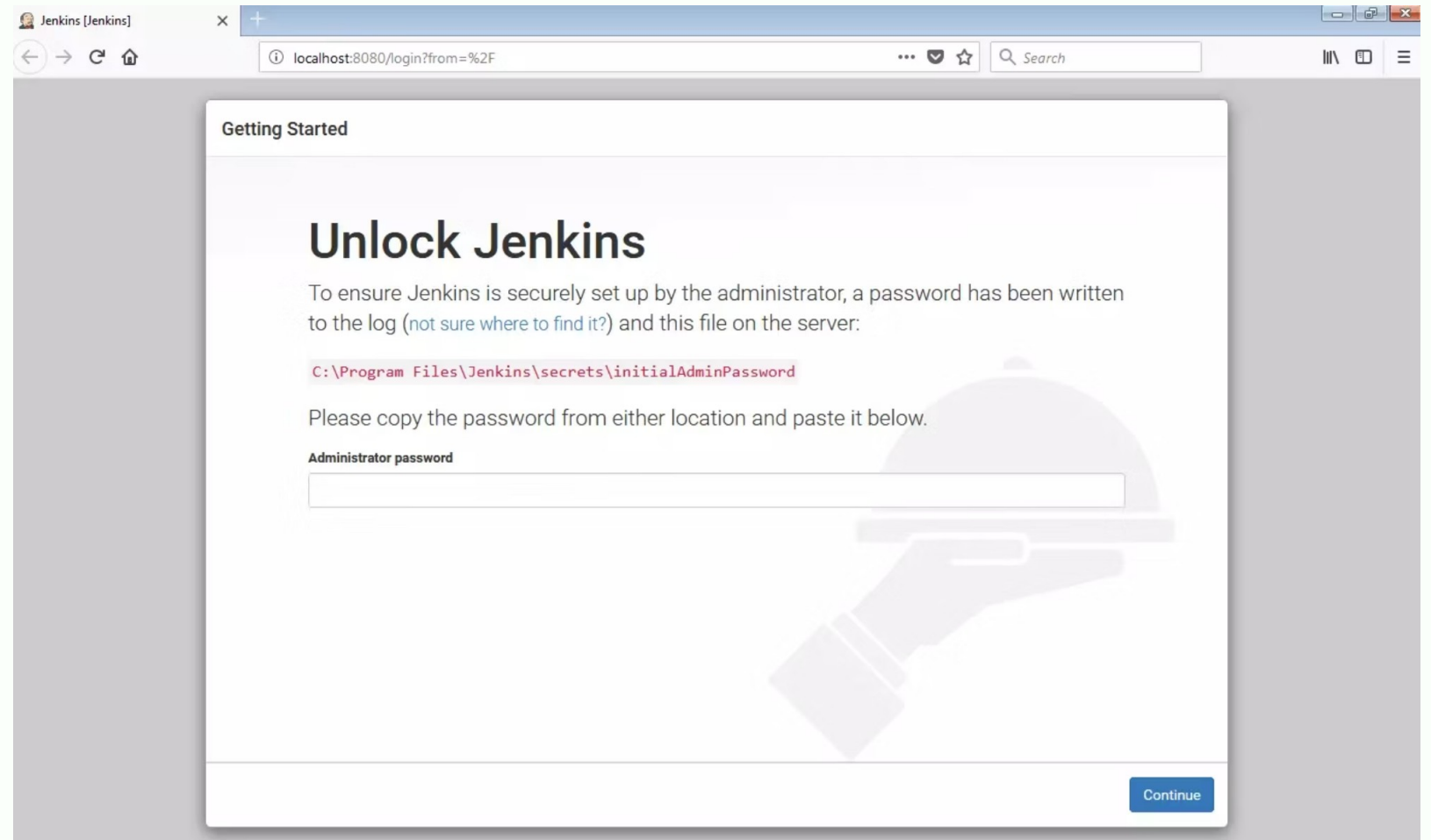
2. Retrieve the initial admin password from the specified file

Choose between "Install suggested plugins" or "Select plugins to install"

4. Wait for plugin installation to complete

5. Create your first admin user with secure credentials

6. Configure the Jenkins URL



Jenkins Dashboard Tour

Main Dashboard Elements

The central hub displays job status, recent builds, and system health

Left Sidebar Navigation

Access to jobs, build history, credentials, and system configuration

Status Icons

Visual indicators show build status (success, failure, in progress)

 **Jenkins**

Search

Dashboard

 New Item

 People

 Build History

 Manage Jenkins

 My Views

 Lockable Resources

 New View

Build Queue

No builds in the queue.

Build Executor Status

1 Idle

2 Idle

Welcome to Jenkins

This page is where your Jenkins jobs live. You can view the status of your builds or start building a software project.

Start building your software

Create a job

Set up a distributed build environment

Set up an agent

Configure a cloud

Learn more about distributed builds

Security Basics

Admin Account Security

- Use strong, unique passwords
- Consider enabling 2FA with appropriate plugins
- Regularly audit admin accounts

Role-Based Access Control

Install the "Role-based Authorization Strategy" plugin

- Create roles with specific permissions
- Assign users to appropriate roles



Implementing proper security measures is crucial for protecting your CI/CD pipeline

Next Steps

Install Essential Plugins

Extend Jenkins functionality with plugins for your specific tech stack and workflow needs.

Set Up Agents

Add build agents to distribute workload and support different environments for your builds.

Configure Your First Pipeline

Create a simple pipeline job to validate your Jenkins setup and understand the basics of pipeline syntax.

Implement CI/CD Best Practices

Configure automated testing, deployment strategies, and notification systems.

Remember: Jenkins is highly customizable. Start with a minimal viable setup and expand as your needs grow.

Post-Installation Checks

1. Environment Variable Configuration

Ensure your Java environment is correctly set up by configuring `JAVA_HOME` and updating your system's `PATH` variable. This is critical for Jenkins and any build tools that rely on Java.

`JAVA_HOME` points to your JDK installation directory.

Add `%JAVA_HOME%/bin` (Windows) or `$JAVA_HOME/bin` (Linux/macOS) to your `PATH`.

- Restart your terminal or system for changes to take effect.

2. Verify Java and Jenkins Runtime

Confirm that Jenkins is using the correct Java version and is running as expected:

Check Java version: `java -version`

Verify Jenkins service status: `sudo systemctl status jenkins` (Linux)

Access the Jenkins web interface at `http://localhost:8080` to ensure it loads correctly.

Proper environment setup prevents common issues and ensures a stable Jenkins operation.

